Extracting wildlife pictures from emails

Anthony LELCHAT

Ostbayerische Technische

Hochschule Regensburg
Regensburg, Germany

anthony.lelchat@extern.oth-regensburg.de

Jan DÜNNWEBER

Ostbayerische Technische

Hochschule Regensburg
Regensburg, Germany

jan.duennweber@oth-regensburg.de



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

Tables of contents

I) Wildlife tracking project 5
1) Why the project is necessary. 5
2) Architecture of the project 7
3) How does the project works. 8
4) Functionalities of the project. 10
a) Determine bears hotspots. 10
b) Determine bears paths 13
II) Mail pictures extraction system 14
1) Accessing the mail inbox 14
a) Approach used 14
b) Difficulty encountered 16
2) Parsing and storing pictures 17
a) Approach used 17
b) Difficulty encountered 19
3) Processing picture from email. 20
4) Visualization on the web interface. 21
a) Approach used 21
b) Difficulty encountered 22
5) Project documentation and dependency management 22
III) Possible Improvement 24
1) Animal detection 24
2) Image analysis 25
Glossary 27

Table of illustrations

Figure 1: Wildlife tracking map 4
Figure 2: Number of people injured/killed by bears over the years. 5
Figure 3: Number of bear sightings and cullings in Japan over the years 6
Figure 4: Example picture of bears detected in the forest. 8
Figure 5: Example mail sent by the cameras. 9
Figure 6: Content of the two HTML files sent by the cameras. 9
Figure 7: First point chosen on the map among the bear sighting point 10
Figure 8: Formula to calculate the abscissa and the ordinate of the next point 11
Figure 9: Second point calculated using the formulas. 11
Figure 10: Process repeated until a hotspot is found 12
Figure 11: Getting the access token for the application 15
Figure 12: OAuth2Credentials config to access the mail inbox 16
Figure 13: Picture of a deer detected by cameras. 17
Figure 14: How easyOCR is used to get the information from a picture 18
Figure 15: Picture information inserted into the database 19
Figure 16: Activity diagram which describe how the process works. 20
Figure 17: How the web interface looks like. 21
Figure 18: Content of the pyproject.toml which define package needed for the project 22
Figure 19: Documentation made using the sphinx documentation package 23
Figure 20: Picture of a man detected as a boar 24
Figure 21: Picture with a man correctly detected 25

Introduction

As a part of my nine-week internship with the Ostbayerische Technische Hochschule, I was asked to work on a feature for an ambitious project consisting of tracking how the wildlife animals move in the area of Fukushima.

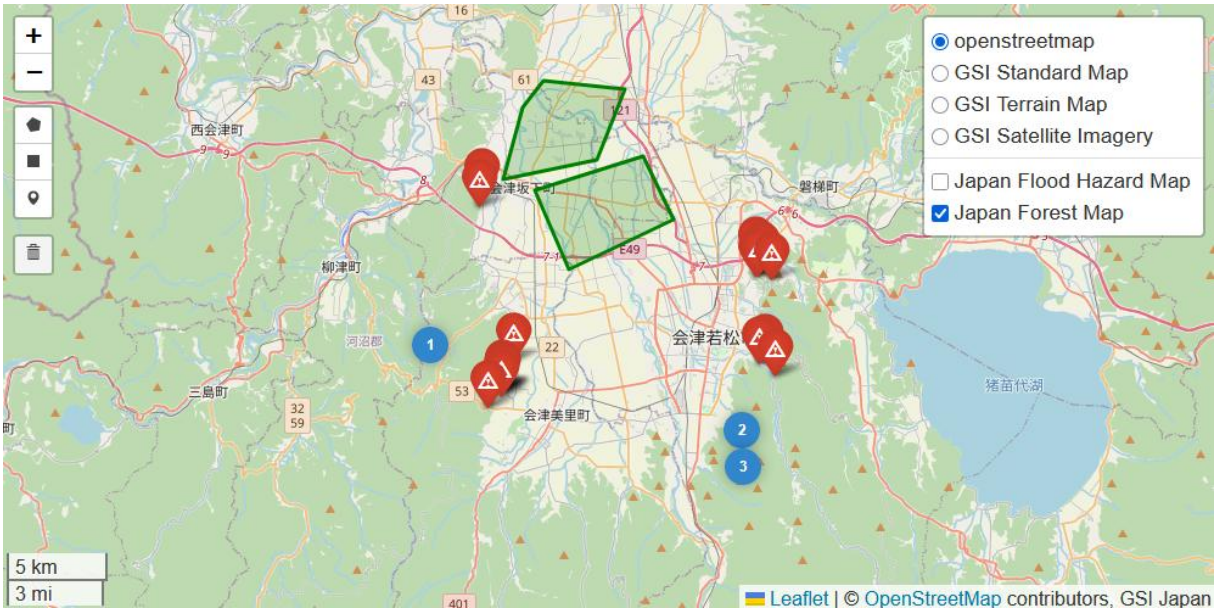


Figure 1: Wildlife tracking map

The feature I had to develop was a program that will extract pictures from the wildlife to store them into a database. Then, I had to connect this database to the web interface. Thanks to that feature, everyone could see the wildlife pictures on the website.

This internship was an opportunity for me to apply what I learned at the ENSSAT Lannion and at Orange. It's also a way for me to improve my English and to discover a different culture.

I will first describe how the project works and why it is needed, and then I will explain what I developed and how my feature is an improvement for the project. The goal of this report is to describe the project, the developed solution, and its contribution to the overall system.

I) Wildlife tracking project

1) Why the project is necessary.

For several years, bear attacks have increased in Japan. These attacks mean that more people are injured or killed. The record for injuries was set in 2014 with 230 people while the record for the number of deaths was set in 2025 with 13 people.

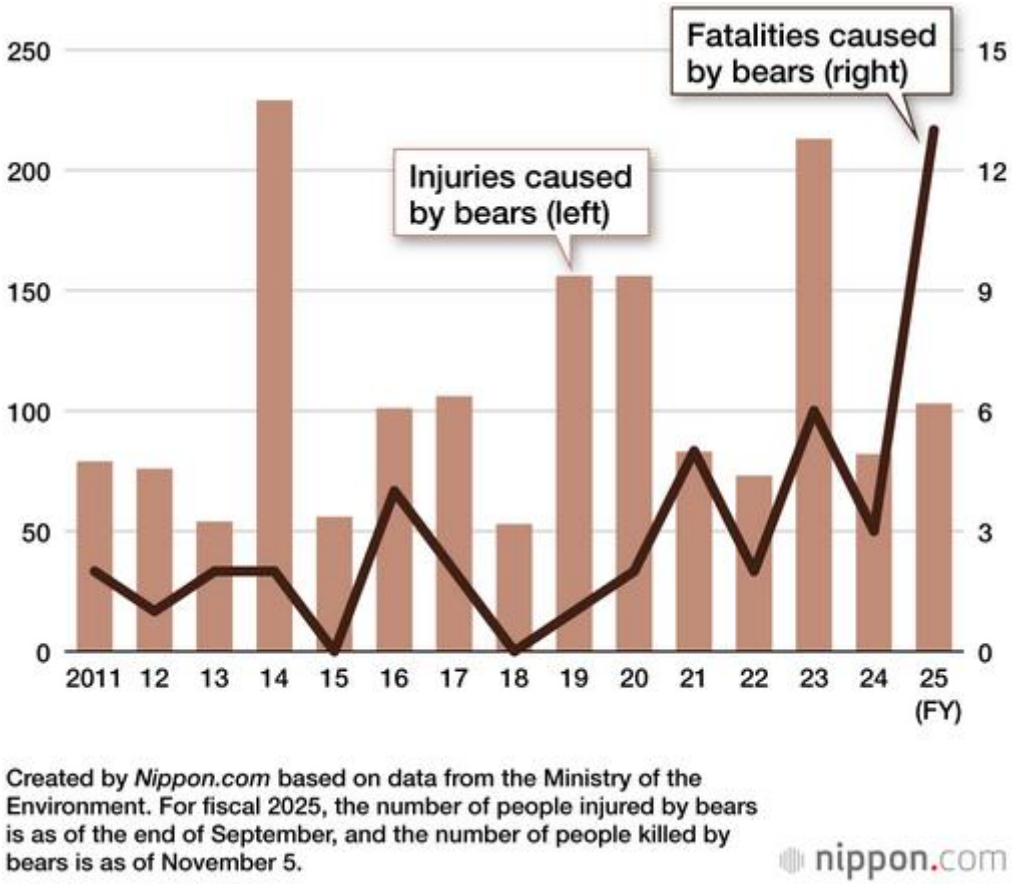


Figure 2: Number of people injured/killed by bears over the years.

The reason for this increase is simple, and it's called climate change. Bears rely heavily on nuts such as acorns and beechnuts. Due to poor harvests caused by climate change, abnormal weather patterns, and forest degradation, these food supplies have become unreliable. When food is scarce, bears are forced to search for alternatives, often leading them into villages, farms, and towns.

Climate change has also disrupted bears' natural cycles, particularly hibernation patterns. Warmer winters shorten hibernation periods or prevent bears from hibernating altogether. As a result, bears remain active for longer periods, increasing the chances of encounters with humans. Extreme weather events also affect plant growth, which reduces the food availability in the wild.

Due to the recent surge in bear attacks, affected municipalities have taken various measures to drive out the intruders. Political considerations include, in many places, the expansion of shooting permits. The record has been set in 2023 with 24000 bear sightings and 9000 bears culled.

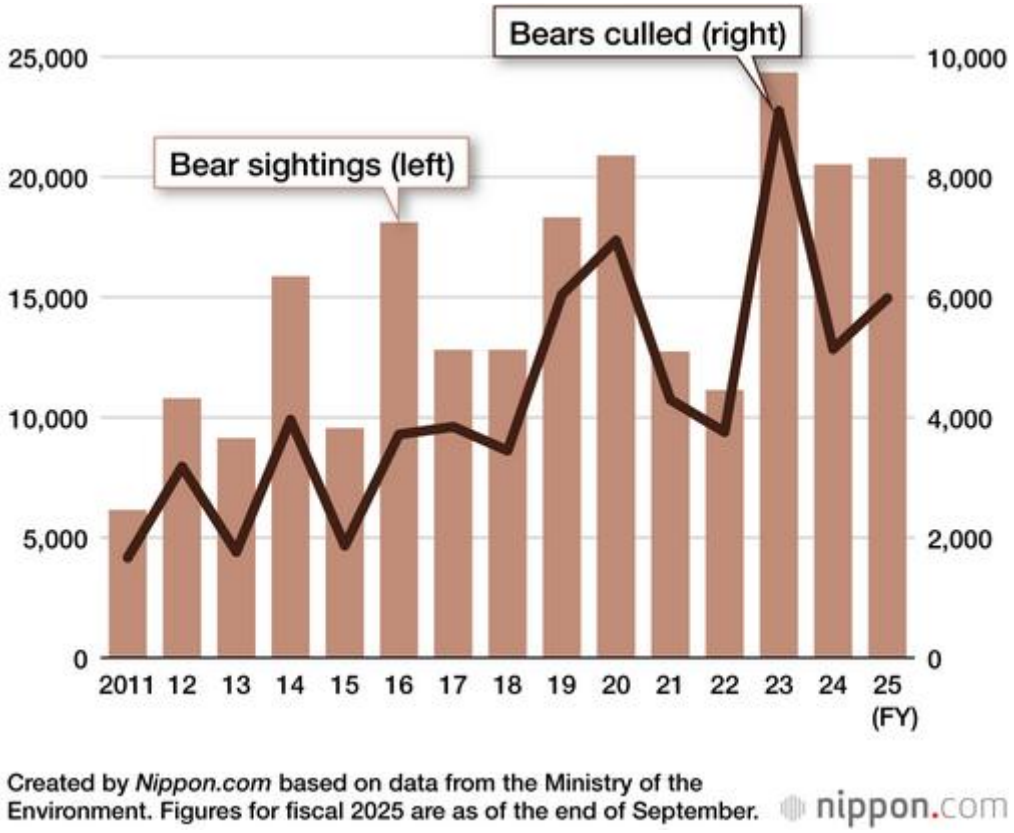


Figure 3: Number of bear sightings and cullings in Japan over the years

From an ecological perspective, such procedures, which are all solely aimed at decreasing the bear population, are questionable, since ecosystems can benefit from a peaceful coexistence of bears with humans. Bears remove animal carcasses and thus return vital nutrients to the soil and prevent the spread of diseases. Moreover, the bears keep the deer population in check, which is advantageous for the forest vegetation.

Conflicts start once the bears intrude populated areas, which are often rural villages and suburbs with a low population density.

In order to decrease these bear attacks, more and more regional governmental authorities invest in technologies for efficient bear hunting, such as IP cameras and computer vision. Cameras can document bear presence but do not actively prevent conflicts, since they merely capture sightings after bears have already entered human areas. Without predictive analytics, authorities lack the ability to anticipate movements, issue early warnings, or strategically deploy deterrents. The goal was to create a proactive, data-driven early warning system that identifies hotspots and forecasts movement routes before encounters occur.

2) Architecture of the project

The wildlife tracking project relies on a clear separation between backend processing, data storage, and frontend visualization, ensuring modularity and ease of maintenance.

The backend of the project is developed in Python. It is responsible for data acquisition and processing. Backend components handle tasks such as retrieving wildlife data, analyzing information, interacting with the database, and providing data to the web interface.

For data storage, the project uses MariaDB, a relational database management system. This database acts as a central component of the system, allowing both backend services and the web interface to access consistent and up-to-date information.

The frontend of the project is implemented using Streamlit, a Python-based framework for building interactive web applications. Streamlit provides a simple and efficient way to visualize wildlife data.

The interaction between the backend, the database, and the frontend follows a straightforward data flow: the backend processes and inserts data into the database, while the Streamlit-based frontend retrieves and displays this data to end users. This architecture allows the project to evolve easily, for example by integrating new data sources, additional analysis modules, or more advanced visualization features.

3) How does the project work.

The tracking project aims at reducing the number of bear attacks. It's based on the concept of "forests digitalization" or "smart forest", which means putting technological devices in a forest.



Figure 4: Example picture of bears detected in the forest.

The project has Raspberry Pi computers connected to wildlife cameras in the forest, which send pictures of the detected animals by email. This allows for real-time forest surveillance.

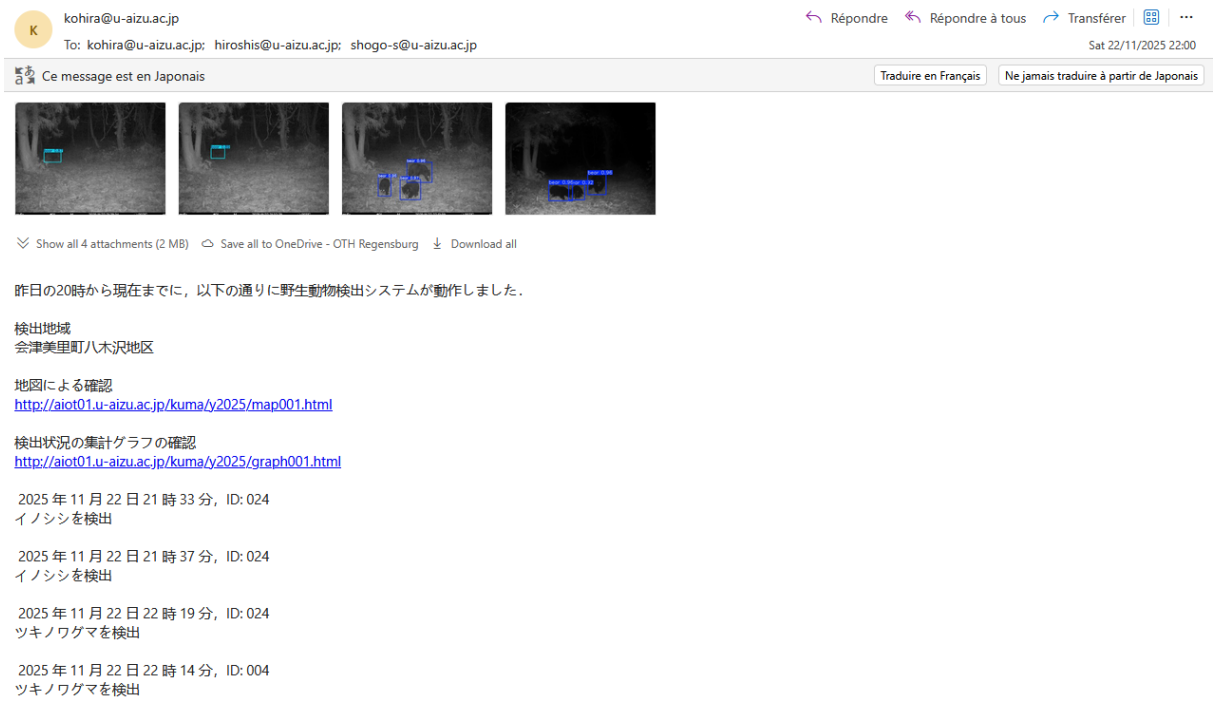


Figure 5: Example mail sent by the cameras.

The mail sent always contains the pictures of the animals detected and two HTML files. One display where the animals are detected on the maps, the other one gives statistics on how many animals have been detected every month and every hour.

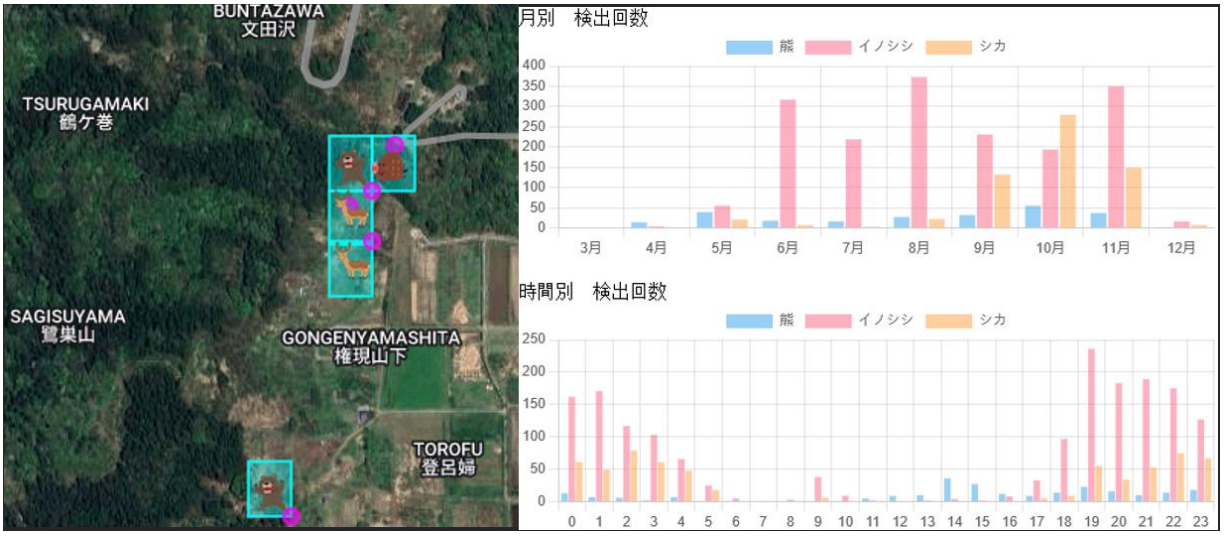


Figure 6: Content of the two HTML files sent by the cameras.

The project controls solar-powered frequency generators equipped with flash lights and speakers to drive off bears from the suburbs back into the forest by scaring them with loud noises and bright light bursts.

4) Functionalities of the project.

a) Determine bears hotspots.

Thanks to all the architecture, the project is able to determine the hotspots of bears. Even if bears are solitary animals, they tend to regroup during spring, summer and autumn to mate, fish or look for a safe place. To determine a hotspot, this feature uses a map containing bear sightings and the algorithm of Weiszfeld. The latter is an iterative procedure for computing the geometric median of a set of points in Euclidean space. The objective is to find a point that minimizes the sum of Euclidean distances to all given points.

Firstly, a point has to be chosen to begin the algorithm, to reduce the number of iterations, a point in the center of the map or near a group of sighting points is preferable.

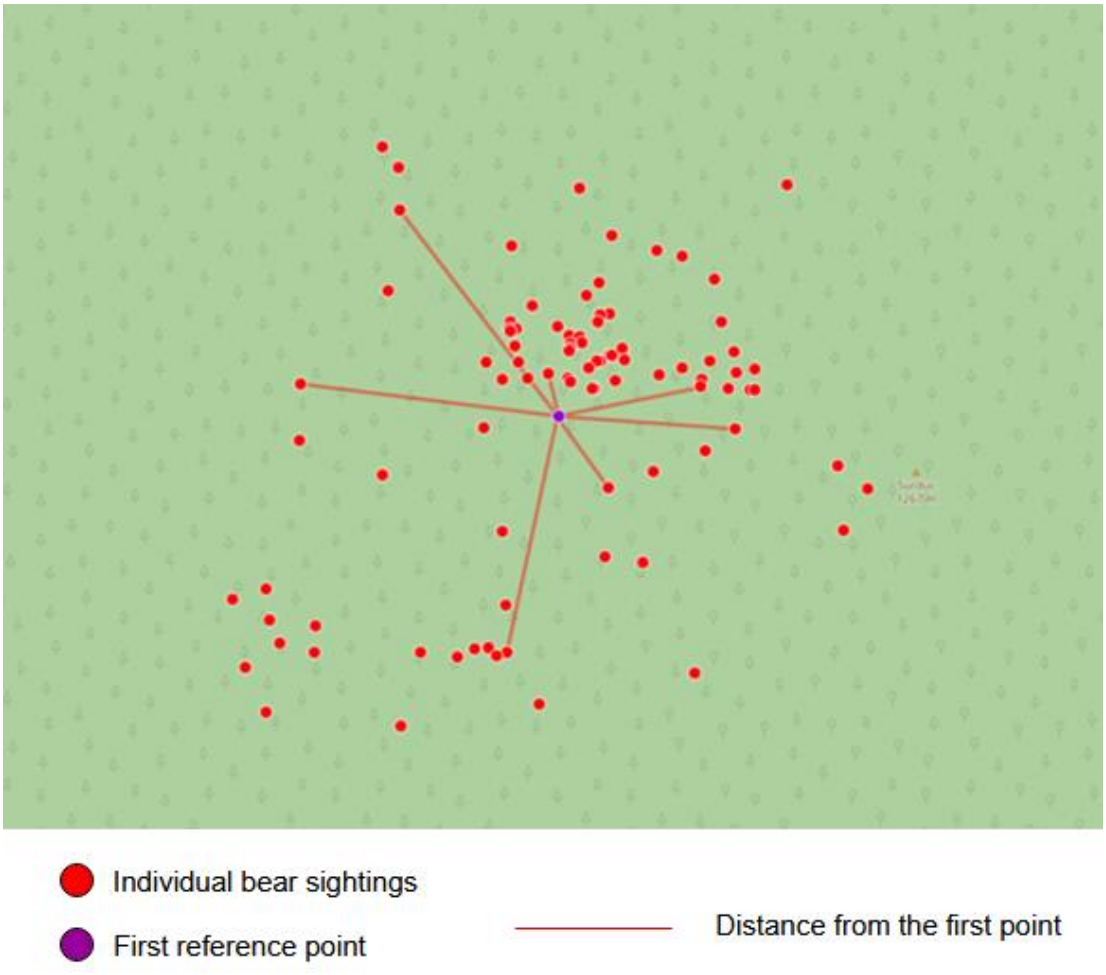


Figure 7: First point chosen on the map among the bear sighting point

Secondly, the algorithm will take every sighting point and will give them a weight, which determines how much the sighting point will influence the next reference point. The weight is equal to one divided by the distance, it means that if the distance from the reference point to a sighting point is high, the sighting point will receive a low weight. The abscissa of the second point is calculated using the weight we got before. It's equal to the sum of every sighting point abscissa multiplied by their weight, divided by the sum of all weights. We follow the same calculation for the ordinate.

$$x_{k+1} = \frac{\sum_{i=1}^{n} \frac{x_i}{d_i}}{\sum_{i=1}^{n} \frac{1}{d_i}} \text{ and } y_{k+1} = \frac{\sum_{i=1}^{n} \frac{y_i}{d_i}}{\sum_{i=1}^{n} \frac{1}{d_i}}$$

Figure 8: Formula to calculate the abscissa and the ordinate of the next point

We now got a new point on the map, which corresponds to the abscissa and the ordinate calculated.

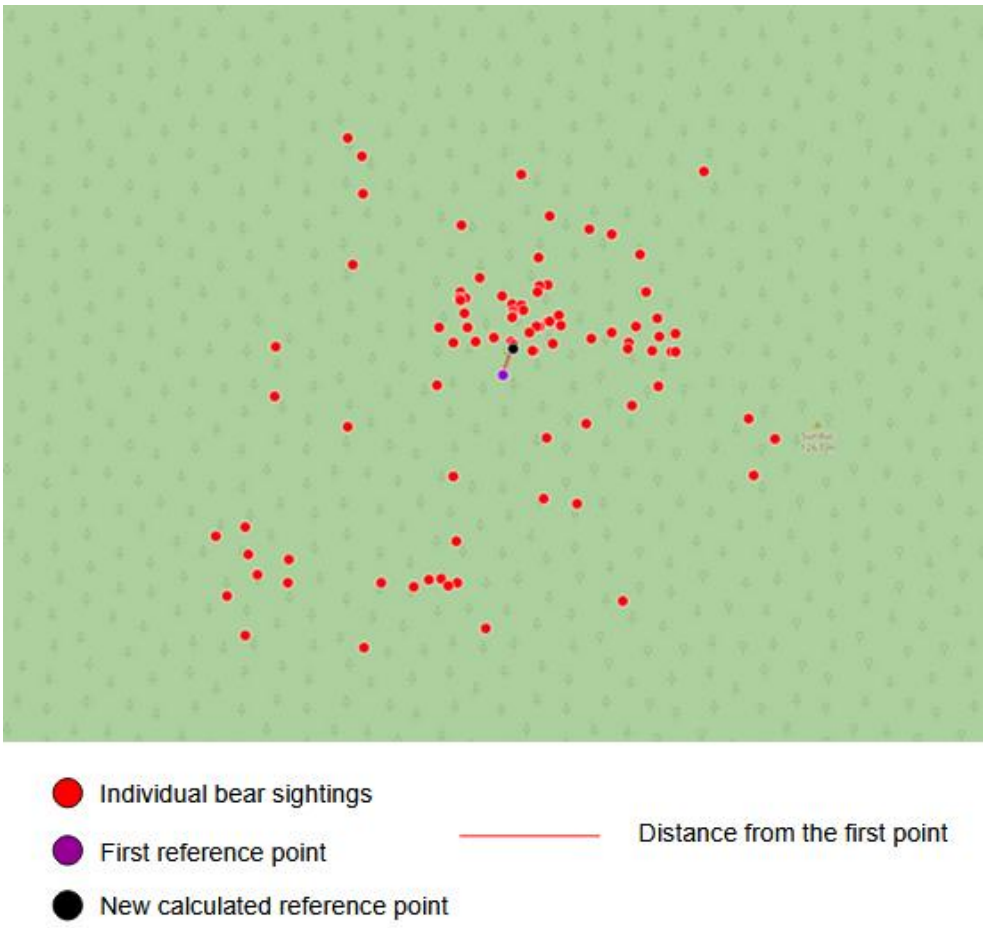


Figure 9: Second point calculated using the formulas.

The process is repeated until the difference of abscissa and ordinate between two reference points becomes insignificant. This last reference point becomes the hotspot point.

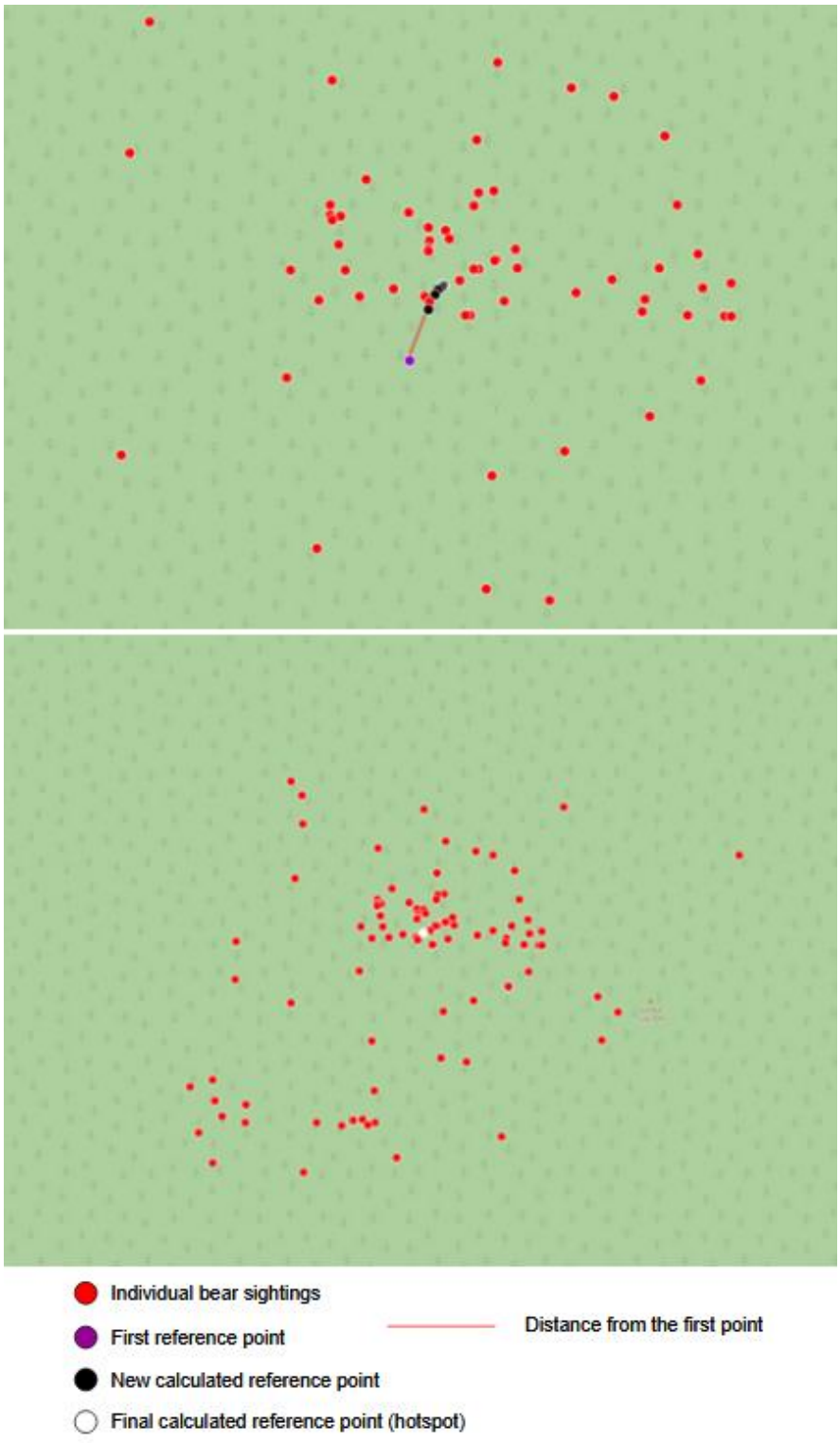


Figure 10: Process repeated until a hotspot is found

b) Determine bears paths

The project can determine the path a bear may take using an AI model. The latter have been trained with GPS data and use the Step Selection Function. This tool evaluates which direction will be most likely taken by bears based on many factors:

- The terrain type (forest, meadow, slope)
- The distance and the angle toward hotspots
- The age and sex of the bears
- The season and the time of the day
- The avoidance of steep or open areas

The AI model has an accuracy of 73% in determining a path that a bear may take.

The two functionalities of the project have been tested on a dataset of brown bears in Romania (13 bears between 2011 and 2015 giving 60000 GPS points) and on a dataset of black bears in Japan (pictures received from cameras). However, one main issue of the project is that, for now, the system that determines what path a Japanese black bear may follow and the hotspots is based on historical data and not on live data. To improve the project, we must have an algorithm that collects pictures sent by email, analyses them to retrieve the relevant data, and stores them into a database. Then, the backend server will be able to access the database in order to use real-time pictures.

II) Mail pictures extraction system

1) Accessing the mail inbox

a) Approach used

The first step of the algorithm is to access the mail inbox. At the OTH, all email addresses are using the Microsoft services. To access the inbox, we need to find an existing package that supports the use of this system. A few packages were considered for the project, but we ultimately chose a package named “Microsoft Graph API”, which is a powerful package maintained and developed by Microsoft to access your email inbox by code. Exchangelib was also considered, and it was the package we chose at first because it was the easiest to use. We finally decided to use Microsoft Graph API for many reasons:

- Exchangelib needs a complex configuration on the Azure Portal compared to the Microsoft Graph API.
- Microsoft Graph API is maintained by Microsoft, whereas Exchange Web Services used by exchangelib aren’t.
- Microsoft Graph API uses a REST (Representational State Transfer) architecture compared to the SOAP (Simple Object Access Protocol) architecture of exchangelib. REST is more suitable for modern, scalable, cloud-based APIs and is more performant, while SOAP fits legacy or highly regulated enterprise systems.
- Because of a lack of security link to the Exchange Web Services not maintained by Microsoft, exchangelib will be deprecated soon.

Another package was dbmail, which doesn’t use the Microsoft Exchange system but can still be a good solution. However, it doesn’t support the shared mail functionalities, which means the package isn’t compatible for the use we want.

```python
class MicrosoftGraphApi:

    def __init__(self):
        """

        Initialize the MicrosoftGraphApi with credentials from environment variables.

        """
        self.tenant_id = os.getenv("TENANT_ID")
        self.client_id = os.getenv("CLIENT_ID")
        self.client_secret = os.getenv("CLIENT_SECRET")
        self.authority = os.getenv("AUTHORITY")
        self.scope = [os.getenv("SCOPE")]

        self.app = msal.ConfidentialClientApplication(
            self.client_id,
            authority=self.authority,
            client_credential=self.client_secret
        )

    def get_access_token(self) -> str | None:
        """

        Acquire an access token for client credentials flow.

        :return: The access token string.
        :rtype: str | None
        :raises Exception: If unable to acquire access token.

        """
        result = self.app.acquire_token_for_client(scopes=self.scope)

        if "access_token" not in result:
            raise Exception("Unable to acquire access token.")

        return result["access_token"]
```

Figure 11: Getting the access token for the application

In order to access the inbox, it's compulsory to retrieve the access token of the Azure application. For that, we used MSAL (Microsoft Authentication Library), which allows to get the access token if all needed information is given (the identifier of the tenant and the client, the client secret…) As Microsoft Graph API needs to be used with HTTP methods (GET, POST, PATCH, DELETE, PUT), we only have to pass the access token in the header of the request to access the inbox.

b) Difficulty encountered

The main difficulty here was in choosing the right package for the project. At first, we chose exchangelib, and there are two ways of accessing the mail inbox through this package: a configuration with a basic authentication (username and password) or a configuration with an OAuth2 authentication (client id, tenant id, and a client secret). The basic authentication is way easier to create and use, but isn't secure compared to an OAuth2 authentication. Since 2017, basic authentication has been deprecated and can't be used anymore. To create an OAuth2 authentication, it's compulsory to create it on the Microsoft Azure portal because it can't be created elsewhere. However, only an administrator can do it.

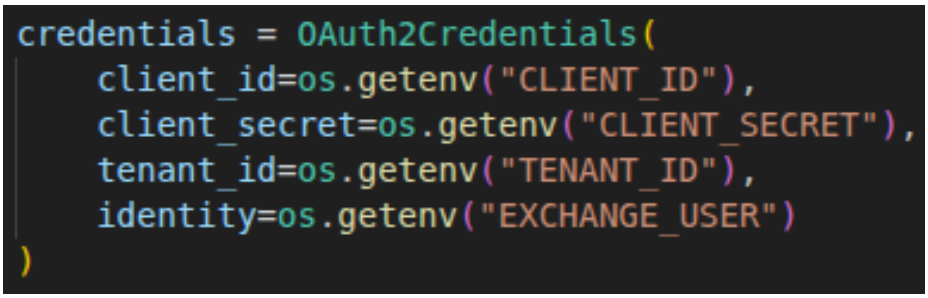


Figure 12: OAuth2Credentials config to access the mail inbox

In order to use my feature in the project, the administrator had to create an application in the Microsoft Azure portal. It gave him all the necessary information listed above to access the inbox. The procedure took some time, and I couldn't try my program during that moment. After some weeks, we got an answer that indicated that the Azure application will be created, but the use of Microsoft Graph API is strongly recommended as exchangelib will be deprecated. I had to take some time in order to understand how the Microsoft Graph API works, as it's a difficult package compared to exchangelib.

2) Parsing and storing pictures

a) Approach used

As we can access the email inbox, we can now collect the pictures sent. The second step of the algorithm is to analyze these pictures.



Figure 13: Picture of a deer detected by cameras.

For every picture, we have different useful information:

- The type of the animals detected.
- A number from 0 to 1 follows ‘deer’, which shows how confident the model is that the animal is a deer.
- The date and the time when the animals have been detected.
- The identifier of the camera that took the picture (in the filename).

In order to get this information, we have to find a package that can analyze pictures. There are a few packages for it, but I used “pytesseract” and “easyOCR”.

At first, we used pytesseract, which is an easy package to use in order to retrieve data in a picture. However, we figured out that pytesseract struggles to get the timestamp and the animal type for pictures. Even with a very detailed and complex configuration, pytesseract will retrieve either nothing or incorrect data. That's why we decided to use easyOCR, which takes more time to analyze a picture, but it successfully retrieves the animal type and the timestamp.

```python
def extract_images_information(self, image_bytes):
    """
    Extract animal type and timestamp from wildlife camera image using EasyOCR.

    :param image_bytes: The binary content of the image.
    :type image_bytes: bytes
    :return: a dict containing information about the image, including whether a bear is detected and the timestamp
    :rtype: dict
    """

    extracted_info = {"has_bear" : False, "timestamp" : None}

    image = Image.open(io.BytesIO(image_bytes))
    ocr_results = self.reader.readtext(image, detail=0)

    for text in ocr_results:
        if "bear" in text.lower():
            extracted_info["has_bear"] = True

        timestamp_match = re.search(r'(\d{4})[/](\d{2})[/](\d{2})\s+(\d{2})[.](\d{2})[.](\d{2})', text)
        if timestamp_match:
            year, month, day, hour, minute, second = timestamp_match.groups()
            extracted_info["timestamp"] = datetime.datetime(
                int(year), int(month), int(day),
                int(hour), int(minute), int(second)
            )

    return extracted_info
```

Figure 14: How easyOCR is used to get the information from a picture

For every picture, the algorithm will get the information (the timestamp and if there is a bear in the picture). Then, it will create a new instance in the database by inserting the information we've collected. Before inserting everything in the project database, I've created my own MariaDB testing database in order to use my algorithm and make sure it works fine. For that, I used DBeaver, which is an open-source database management tool that allows users to connect to, explore, and manage multiple types of databases through a graphical interface.

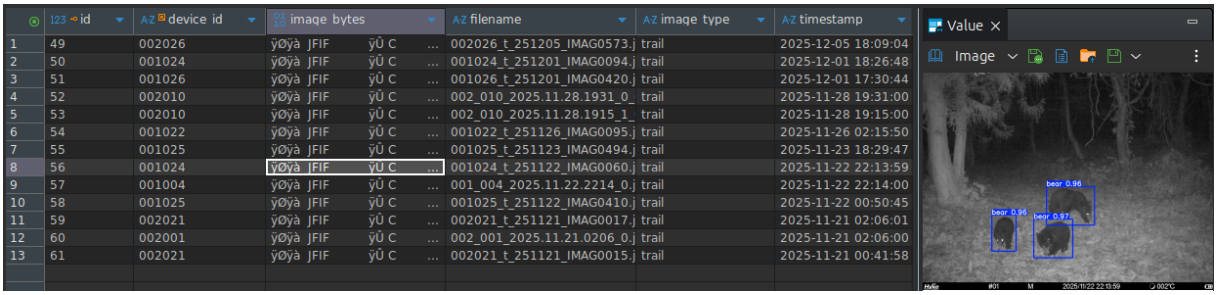


Figure 15: Picture information inserted into the database

After inserting information from a picture into the database, we can see that we have a field for the identifier, the device identifier, the metadata content of it (it allows us to see the picture in DBeaver), the name of the attachment, the type of camera that took the picture, the timestamp, and finally, a field “parsed_successfully”, which is true when the camera identifier and the timestamp have been correctly retrieved.

b) Difficulty encountered

The main difficulty with this task was in the packages pytesseract and easyOCR. I’ve never used these packages before, it took me some time to understand how I could use them in the algorithm, and to figure out how I could efficiently retrieve the timestamp and the animal type from a picture. I was always trying to improve my configuration of pytesseract in order to finally get a convenient result. Using easyOCR for the project made it easier to retrieve data.

3) Processing picture from email.

After having an algorithm that can access an inbox, analyze a picture, and store it into a database, we can now develop the whole processing system.

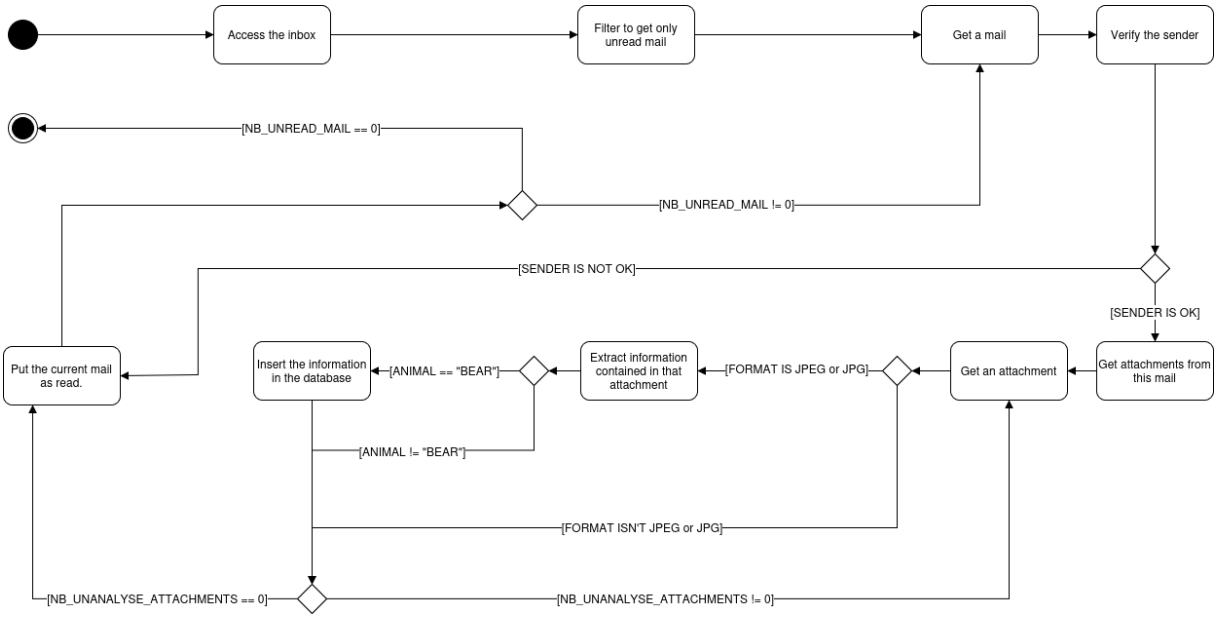


Figure 16: Activity diagram which describe how the process works.

The process part is simple because it's using every algorithm we developed. First, it will access the inbox and filter to only get unread emails (we consider that read emails are emails that have already been treated). Then, it will get an email and determine if the sender is the Aizu project address. After that, it will collect all the attachments from the mail. Then, I will get an attachment and collect its data if it's a JPEG or JPG file. These data will be inserted into the database only if there is a bear in the picture. When an attachment is parsed, we take another picture from the email. If the mail has been completely analyzed, we put it as "read". The process is repeated until all pictures and email have been parsed.

4) Visualization on the web interface.

a) Approach used

The final goal of the project was to be able to visualize the real-time data we now got through the project in the web interface. There is already a developed web interface project, but it was using historical data. The project was using a different method of storing pictures into the database: The path of the picture was in the database, but the image bytes themselves weren't. The project had to retrieve the path in the database to find it into the file system of the server. We asked me to change the database in order to directly store the picture bytes in the database, and to be able to retrieve them into the web interface. In order to see the pictures automatically inserted in the database, I had to change the database used by the web interface in order to store a BLOB (Binary Large Object) and not a path. I also changed a function in the web interface in order to directly display the pictures when the user tries to load those who are associated with the selected camera identifier.

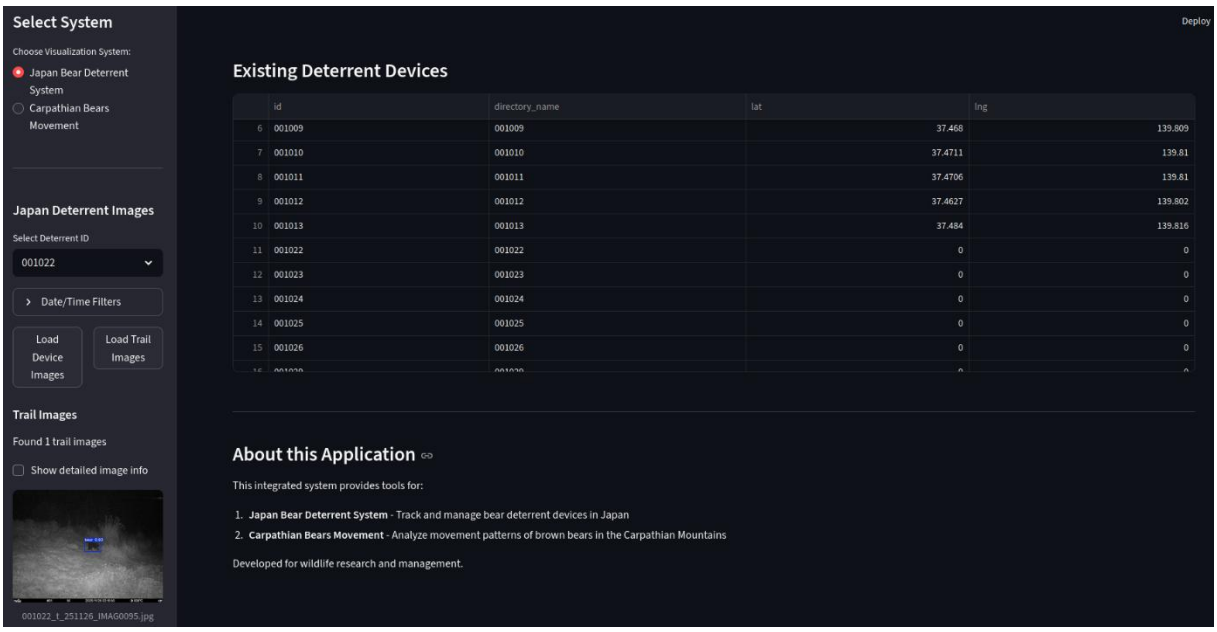


Figure 17: How the web interface looks like.

We finally get this, which is exactly what we wanted: The extracted pictures are parsed and stored in the database, and they are finally displayed on the web interface.

b) Difficulty encountered

The main difficulties of this task were to understand the code of the web interface and make changes to the function without breaking any other features provided by the project. I had to understand what the previous database looked like in order to create something similar. The project is also using Streamlit, which is a package that allows the creation of web interfaces in Python. It was a package I had never used before, and I took some time to understand it.

5) Project documentation and dependency management

During the development of the algorithm, I used a package named “poetry” to manage dependencies. The only thing you have to worry about is the version of Python you are using. By choosing a compatible version of Python, you won’t have any issue with the package used.

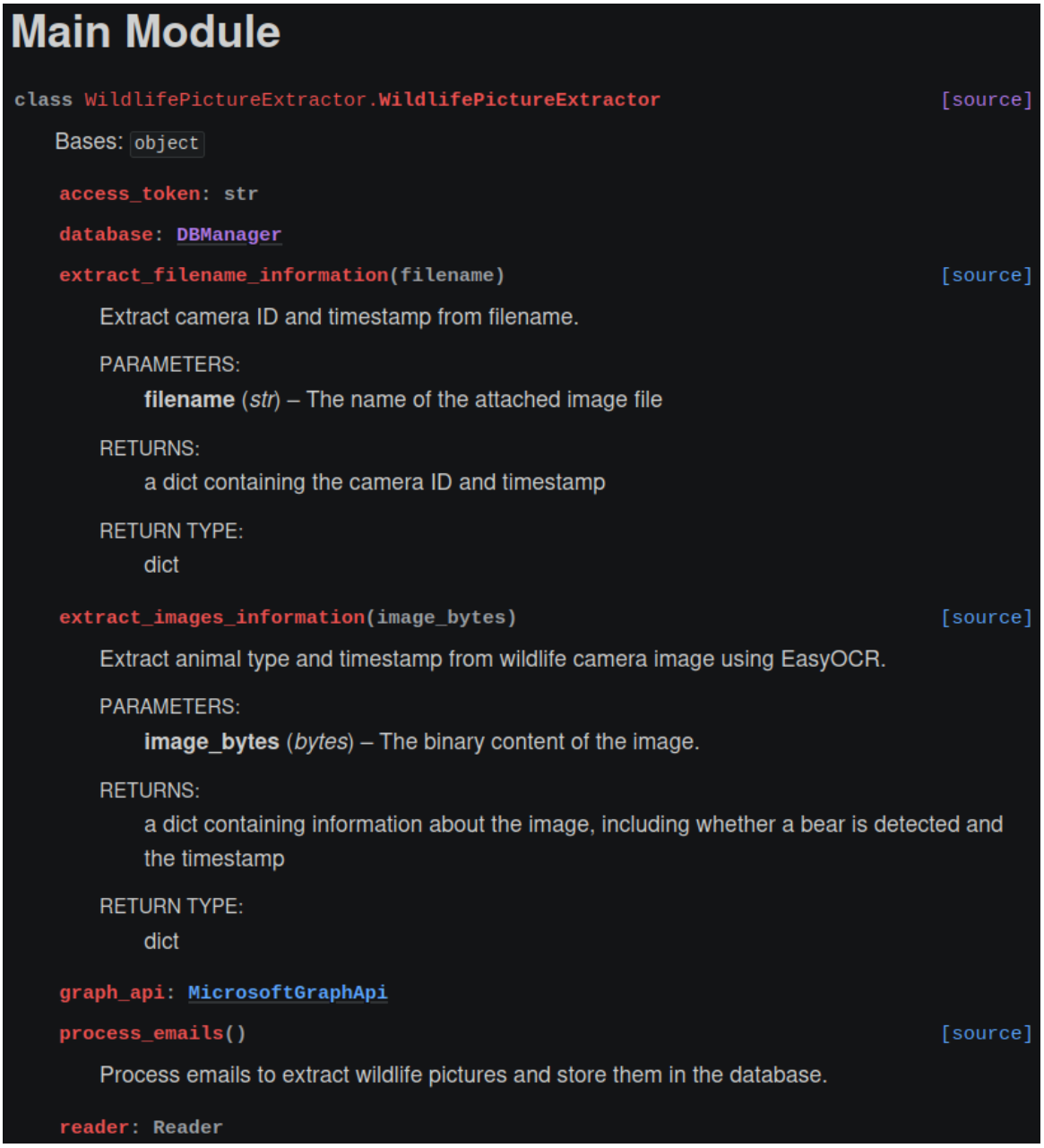
```
[project]
name = "wildlife-pictures-extraction"
version = "0.1.0"
description = "package that aims at collecting pictures in email to analyse them
authors = [
    {name = "Anthony Lelchat",email = "anthony.lelchat@extern.oth-regensburg.de"}
]
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "mysql-connector-python (>=9.5.0,<10.0.0)",
    "sphinx (>=9.1.0,<10.0.0)",
    "dotenv (>=0.9.9,<0.10.0)",
    "pytesseract (>=0.3.13,<0.4.0)",
    "furo (>=2025.12.19,<2026.0.0)",
    "msal (>=1.34.0,<2.0.0)",
    "requests (>=2.32.5,<3.0.0)",
    "easyocr (>=1.7.2,<2.0.0)",
]

[tool.poetry]
package-mode = false

[build-system]
requires = ["poetry-core>=2.0.0,<3.0.0"]
build-backend = "poetry.core.masonry.api"
```

Figure 18: Content of the pyproject.toml which define package needed for the project

For example, if you use Python 3.12 or a newer version to use the feature, doing a “poetry install” will directly create a virtual environment and will install in the latter all needed packages for the project, considering the version of Python. Without that, installing the latest version of every package needed could make the program to fail.



Main Module

class WildlifePictureExtractor.**WildlifePictureExtractor** [source]

Bases: `object`

access_token: str

database: DBManager

extract_filename_information(filename) [source]

Extract camera ID and timestamp from filename.

PARAMETERS:
filename (*str*) – The name of the attached image file

RETURNS:
a dict containing the camera ID and timestamp

RETURN TYPE:
dict

extract_images_information(image_bytes) [source]

Extract animal type and timestamp from wildlife camera image using EasyOCR.

PARAMETERS:
image_bytes (*bytes*) – The binary content of the image.

RETURNS:
a dict containing information about the image, including whether a bear is detected and the timestamp

RETURN TYPE:
dict

graph_api: MicrosoftGraphApi

process_emails() [source]

Process emails to extract wildlife pictures and store them in the database.

reader: Reader

Figure 19: Documentation made using the sphinx documentation package

I also created documentation for the project, one in markdown to describe what the project is doing and another one using the Sphinx packages. The latter creates a HTML documentation using the docstrings done in all methods and classes.

III) Possible Improvement

1) Animal detection



Figure 20: Picture of a man detected as a boar

Sometimes, the cameras fail to guess the type of the “animal” detected. Here, it detected a boar, but it’s a man. One possible explanation could be that the model struggles with types that don’t exist. However, since the ‘man’ type exists, this issue highlights limitations in the detection accuracy rather than missing labels.

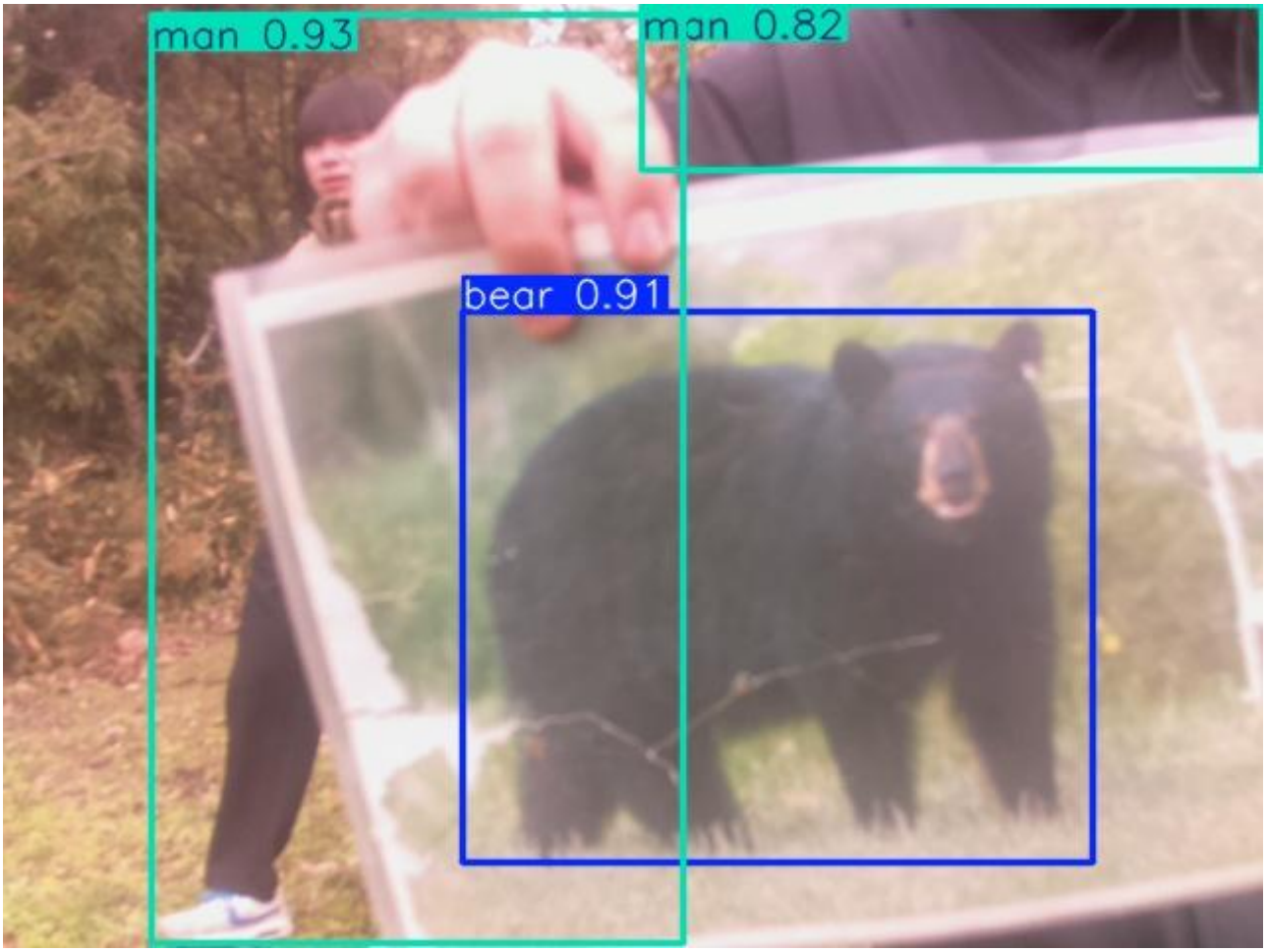


Figure 21: Picture with a man correctly detected

Maybe in the future there will be some improvement in order to correctly identify the type of animal detected because these types of errors could create inconsistencies in the database and on the web interface.

2) Image analysis

Sometimes, there are some pictures (like Figure 21) that don't have any date and time in hem. This could lead easyOCR and our image parsing algorithm to fail to get some information. A possible reason for this issue could be that old cameras don't correctly display the date and the time on the pictures they took. However, it's not a critical issue, we figured out that, for all of these pictures without any timestamp, the latter is always stored into the filename.

Conclusion

This internship at the Ostbayerische Technische Hochschule in Regensburg allowed me to actively contribute to a project aimed at reducing human–wildlife conflicts through the use of modern technologies. The wildlife tracking project addresses a real ecological and social issue by combining forest digitalization, computer vision, and data analysis to better understand animal movements and prevent dangerous encounters with humans.

During this internship, I developed a system capable of automatically extracting wildlife pictures sent by email, analyzing them to retrieve relevant information, and storing this data in a database. This feature represents an important improvement for the project, as it enables the backend server to access near real-time data instead of relying solely on historical records. Such an enhancement can increase the accuracy of animal path prediction and improve the effectiveness of preventive measures.

On a personal level, this internship was a valuable opportunity to apply the knowledge I gained at the ENSSAT Lannion and at Orange, to improve my technical skills, and to work in an international and academic environment. It also helped me strengthen my English skills and gain insight into a different culture. Overall, this experience was both professionally and personally enriching.

Glossary

AI model (Artificial Intelligence model)

A computational model trained on data to make predictions or decisions. In this project, an AI model is used to predict potential bear movement paths based on GPS data and environmental factors.

Azure Portal

A web-based management interface provided by Microsoft to configure cloud services such as applications, authentication, and permissions used by Microsoft Graph API.

BLOB (Binary Large Object)

A data type used in databases to store large binary data such as images. In this project, BLOBs are used to store wildlife pictures directly in the database.

Camera identifier

A unique identifier associated with a wildlife camera, usually embedded in the image filename, allowing pictures to be linked to a specific camera location.

Computer vision

A field of artificial intelligence that enables computers to interpret and analyze visual data such as images. It is used here for animal detection and image analysis.

Database

An organized collection of structured data. The project uses a relational database (MariaDB) to store extracted image data and metadata.

Dependency management

The process of handling external libraries required by a software project. Poetry is used in this project to manage Python dependencies and ensure version compatibility.

EasyOCR

An optical character recognition (OCR) library used to extract textual information such as timestamps and animal labels from images.

Exchangelib

A Python library used to access Microsoft Exchange mailboxes. It was initially considered but replaced by Microsoft Graph API due to maintenance and security concerns.

Geometric median

A point that minimizes the sum of distances to a set of points. It is computed using the Weiszfeld algorithm to determine bear hotspots.

Hotspot

A geographical area with a high concentration of bear sightings. Hotspots are calculated from historical and real-time data to identify areas of increased bear activity.

MariaDB

An open-source relational database management system used for testing and storing extracted picture data.

Metadata

Additional information describing data content. Image metadata includes timestamps, camera identifiers, and detection results.

Microsoft Graph API

A RESTful API provided by Microsoft to access Microsoft services such as emails. It is used in this project to retrieve emails and attachments programmatically.

MSAL (Microsoft Authentication Library)

A library used to authenticate applications and obtain access tokens required to use Microsoft Graph API.

OCR (Optical Character Recognition)

A technology that converts text present in images into machine-readable text. OCR is used to extract timestamps and detection information from wildlife pictures.

Poetry

A Python tool used for dependency management and virtual environment creation, ensuring reproducible project setups.

Predictive analytics

Techniques that analyze data to predict future events. In this project, predictive analytics are used to anticipate bear movement paths and identify risk areas.

REST API (Representational State Transfer API)

A web service architecture based on HTTP methods such as GET and POST. Microsoft Graph API follows this architecture.

SOAP (Simple Object Access Protocol)

A messaging protocol for web services. It is compared with REST in the report as a more rigid and legacy-oriented alternative.

Step Selection Function (SSF)

A statistical method used to model animal movement by evaluating possible directions based on environmental and biological factors.

Streamlit

A Python framework used to build web interfaces. It is used in this project to display wildlife pictures and related data.

Virtual environment

An isolated Python environment containing specific versions of dependencies. Created automatically by Poetry in this project.

Weiszfeld algorithm

An iterative algorithm used to compute the geometric median of a set of points. It is applied to calculate bear hotspots.

Wildlife tracking

The monitoring and analysis of animal movements using technological tools such as cameras, GPS data, and data analysis algorithms.